

고급 강화학습 프로젝트

온실 내 로봇 주행을 강화학습으로 풀어보기

25113021 신동석

코드 깃허브 주소 : https://github.com/elasmobranches/2026-1_RL_PROJECT

1. 해결하고자 하는 문제 정의

1.1 연구 배경 및 동기

농업 환경에서 자율 로봇은 방제, 수확, 예찰 등 다양한 작업을 수행할 것으로 기대된다. 그러나 로봇이 각 작업을 수행하는 과정에는 다양한 의사결정 요소가 존재한다. 어느 구역을 먼저 돌 것인지, 어떤 순서로 레인을 방문할 것인지, 작물 상태를 확인하고 어떤 조치를 취할 것인지 등, 모든 결정이 전체 작업 효율을 좌우한다. 기존에는 이런 경로·순서 결정을 사람이 설계한 휴리스틱으로 처리해왔다. 휴리스틱은 잘 설계되면 효과적이지만, 문제마다 적절한 규칙을 사람이 직접 고안해야 하고, 작물 상태를 미리 알 수 없는 부분 관측 환경에서 탐색과 조치의 순서를 명시적 규칙으로 짜기는 까다로울 수 있다.

본 프로젝트는 명시적 수행 규칙을 프로그래밍하지 않고, 보상과 관측 설계만으로 로봇이 효율적인 작업 정책을 스스로 학습하도록 하는 것을 목표로 한다. 즉, 정밀농업 로봇의 작업 수행 정책을 강화학습 문제로 정의하고, 이를 위한 커스텀 환경을 설계·적용한다.

1.2 문제 정의

본 연구에서 농업용 자율 로봇은 다음의 세 가지 핵심 조건을 동시에 만족해야 하며, 이를 달성하기 위한 구체적인 수행 방법에는 다양한 선택지가 존재한다.

조건	설명	의사결정 요소
충돌 없는 이동	작물 구역·벽과 충돌하지 않고 주행 레인만 이동	어느 레인을 경유할지
전체 예찰	모든 작물 셀을 빠짐없이 예찰	방문 순서·경로 패턴
상태 기반 조치	예찰 결과에 따라 정상/수확/방제 구분 후 적절한 액션	조치 타이밍·우선순위

이러한 문제 특성을 고려할 때, 본 문제는 다음과 같은 이유로 강화학습을 통한 해결 가능성을 실험을 통해 보고자 하였다.

첫째, 매 스텝의 이동 선택이 이후 도달 가능한 영역과 잔여 작업량에 영향을 미치는 순차적 의사결정(Sequential Decision Making) 문제이므로 장기 누적 보상을 최대화하는 강화학습 프레임워크와 부합한다.

둘째, 온실 내 작물 상태는 예찰을 수행하기 전까지 완전히 관측되지 않으며, 에이전트는 제한된 정보만을 기반으로 의사결정을 수행해야 한다. 따라서 본 문제는 부분 관측 환경 (Partially Observable Environment)의 특성을 가지며, 강화학습은 이러한 불확실성 하에서 효과적인 탐색 전략을 학습할 수 있다.

셋째, 최적의 예찰 경로는 작물 상태와 발견된 이상 개체의 분포에 따라 매 에피소드마다 달라진다. 따라서 에이전트는 현재까지 획득한 정보와 잔여 예찰 영역을 고려하여 방문 순서를 동적으로 결정해야 하며, 이는 상태 공간이 빠르게 증가하는 조합 최적화 문제의 특성을 가진다. 강화학습은 이러한 환경 변화에 적응하며 효율적인 정책을 학습할 수 있다.

1.3 연구 질문 및 검증 설계

본 연구는 앞서 정의한 세 가지 조건을 만족하는 환경을 구현하고, 알고리즘의 복잡도를 단계적으로 확장하며 다음 네 가지 연구 질문을 검증한다.

Step 1. 단일 정책(Flat Policy)이 이산 격자 환경에서 동적 행동 마스킹(Action Masking)을 활용하여 모든 예찰 구역을 중복 없이 방문하는 정책을 종단간(End-to-End)으로 학습할 수 있는지 검증한다.

Step 2. 계층적 강화학습(Hierarchical Reinforcement Learning)의 효과를 평가한다. 의사결정을 ‘목표 구역 선정(상위 정책)’과 ‘구역 내 이동(하위 정책)’으로 분리했을 때, 단일 정책 대비 최종 성능과 정책 안정성이 어떻게 변화하는지 분석한다.

Step 3. Step 2와 동일한 계층형 의사결정 구조를 유지하되, 상위 정책의 상태 표현과 학습 방식을 확장한 개선 단계를 평가한다. 구체적으로 상위 관측 상태에 미방문 구역까지의 거리 정보를 추가하고 Off-policy 기반 학습을 적용했을 때, 에이전트가 거리 기반 우선 탐색에 가까운 정책을 학습하는지 검토한다.

Step 4. 환경을 연속 공간으로 확장했을 때의 강화학습 알고리즘 특성을 분석한다. 이산 격자 환경을 연속 상태 및 행동 공간으로 변환한 후, SAC, TD3, TQC의 학습 안정성, 수렴 속도 및 최종 성능을 비교 평가한다.

2. 강화학습 환경 설계

본 절에서는 1.2에서 다룬 문제를 담아낼 시뮬레이션 환경의 구조와 동역학, 그리고 이를 마르코프 결정 과정으로 형식화한 상태·행동·보상·종료의 정의를 기술한다. 이 절에서 정의하는 요소는 모든 학습 단계가 공유하는 공통 기반이며, 단계별로 추가·변형되는 부분은 해당 알고리즘과 함께 설명한다.

2.1 맵 구조

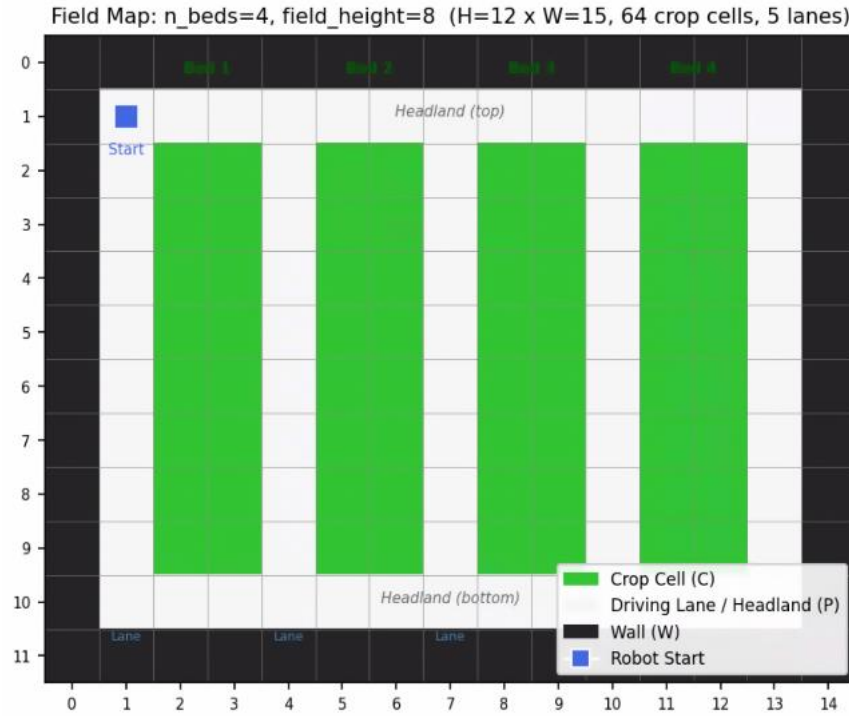


Figure 1. 온실 구조 모식화 격자 맵.

현실의 온실 수직재배 구조를 모델링하기 위해 2열 재배단 격자 맵(Fig. 1)을 사용한다. 토마토 및 파프리카와 같은 온실 작물은 하나의 줄기를 2~3개로 유인하여 수직 재배하는 것이 일반적이며, 이 과정에서 잎과 과실이 줄기 양측으로 넓게 분포한다. 따라서 하나의 작물 줄이 물리적으로는 단일 개체라 하더라도, 로봇이 특정 레인에서 접근할 경우 모든 작업 대상에 도달할 수 없으며 일부 영역은 반대편 레인에서만 접근이 가능하다. 본 연구에서는 이러한 구조적 제약을 반영하기 위해 단일 작물 줄을 안쪽 열과 바깥쪽 열로 분리한 2열 구조로 추상화하였다. 이에 따라 전체 작업 영역을 포괄하기 위해서는 인접한 두 레인을 모두 경유해야 한다.

각 작물은 단순한 정적 객체가 아니라 상태 정보를 가지는 확률적 객체로 정의된다. 에피소드 시작 시 각 작물의 초기 상태는 사전 정의된 확률 분포(Table 1)에 따라 무작위로 할당되며, 해당 상태는 예찰이 수행되기 전까지 에이전트에게 관측되지 않는다. 따라서 에이전트는 제한된 정보만을 바탕으로 의사결정을 수행해야 하며, 본 환경은 부분 관측 마르코프 결정 과정(POMDP)으로 모델링된다.

Table 1 작물의 초기 상태 확률 분포 및 종결 조건

초기 상태	확률	종결 조건
정상 (1)	60%	예찰만으로 확인 완료
수확 필요 (2)	25%	수확 액션을 통해 수확 완료 상태(4)로 전이
방제 필요 (3)	15%	방제 액션을 통해 방제 완료 상태(5)로 전이

2.2 상태·행동·보상·종료의 정의

2.1절의 환경을 강화학습 문제로 최적화하기 위해 사용한 관측·행동·보상·종료 조건의 기본 구성은 다음과 같다. 본 설정은 Step 1~3의 이산 환경을 기준으로 하며, 연속 환경(Step 4)에서의 차이점은 2.3절에서 별도로 설명한다.

상태 공간(State Space)은 12×15 격자 맵에 대해 4개의 채널을 구성한 후 이를 평탄화한 720차원 벡터로 정의된다(Table 2). 각 채널은 맵 구조, 에이전트 위치, 예찰 여부, 예찰 결과를 나타낸다. 특히 ch_3 에서는 예찰되지 않은 작물을 0(미지)으로 표현하여 관측 전까지 상태 정보를 제공하지 않는다. 따라서 에이전트는 이후의 작업 보상을 획득하기 위해 예찰 행동을 수행하여 환경 정보를 수집해야 한다.

행동 공간(Action Space)은 4개의 이동 행동과 3개의 작업 행동으로 구성된 *Discrete*(7)공간으로 정의된다(Table 3). 또한 동적 행동 마스킹(Dynamic Action Masking)을 적용하여 현재 상태에서 수행 불가능한 행동을 선택하지 못하도록 제한한다. 구체적으로 유효하지 않은 행동의 로짓(Logit)에 $-\infty$ 에 근사한 매우 작은 값을 부여하여 Softmax 확률을 사실상 0으로 만들고, 이를 통해 무의미한 탐색으로 인한 샘플 낭비를 방지한다. 보상 함수(Reward Function)는 핵심 작업(수확, 방제)에 높은 양의 보상을 배정하고, 일반 이동에 미세한 페널티를 주어 맹목적인 배회를 억제하도록 단일 스칼라 형태로 균형을 맞추어 설계하였다(Table 4). 수확은 경제적 가치가 가장 직접적으로 반영되는 작업이므로 가장 큰 보상을 부여하였으며, 방제 역시 작물 상태 개선을 위한 핵심 작업으로 설정하였다. 또한 신규 예찰과 정상 확인에 보상을 부여하여 에이전트가 단순히 이동하는 것이 아니라 미관측 작물 상태를 확인하도록 유도하였다.

Table 2 이산 환경의 상태 공간 정의

채널	내용	정규화	값
ch0	맵 구조	$\div 2$	0:통로, 1:작물, 2:벽
ch1	에이전트 위치	0/1	현재 위치:1. 나머지:0
ch2	예찰 완료 마스크	0/1	예찰이 완료된 셀: 1 미예찰 셀: 0
ch3	예찰 결과	$\div 5$	0: 미지, 1~3: 작물 상태, 4: 수확 완료, 5: 방제 완료

Table 3 행동 공간 및 동적 마스킹 조건

인덱스	액션	마스킹 조건
0~3	이동 (상/하/좌/우)	목적지가 작물 또는 벽인 경우
4	예찰 (Scout)	인접한 미예찰 작물 셀이 없는 경우
5	수확 (Harvest)	인접한 수확 대기 작물 셀이 없는 경우
6	방제 (Pest Control)	인접한 방제 대기 작물 셀이 없는 경우

Table 4 기본 이산 환경의 보상 함수

이벤트	보상	설계 근거
매 스텝	-0.1	불필요한 배회 억제
충돌 시도	-2.0	안전 내비게이션 유도
신규 셀 예찰	+1.0	탐색 유인
정상 확인	+0.5	예찰 행동에 대한 추가 인센티브
수확 성공	+10.0	핵심 작업 수행
방제 성공	+8.0	핵심 작업 수행
전체 완료	+20.0	전체 작업 영역 커버리지 완성 유도

2.3 단계별 환경 변형 및 알고리즘 선택

1.3절에서 제기한 연구 질문에 답하기 위해, 앞서 정의한 공통 환경을 기반으로 정책 구조와 상태 표현을 단계적으로 확장하였다. 각 단계는 직전 단계에서 확인된 한계를 개선하기 위해 설계되었으며, 이를 통해 단일 정책 기반의 이산 제어에서 계층적 의사결정 구조를 거쳐 연속 제어 환경까지 확장 가능한지를 검증하고자 한다.

2.3.1 Step 1 — 이산 강화학습 (Maskable PPO)

본 단계는 단일 정책(Single Policy)이 경로 계획(Path Planning)과 작업 수행(Task Execution)을 동시에 학습할 수 있는지 검증하기 위한 기준(Baseline) 단계이다. 환경은 2.1~2.2절의 공통 이산 환경(FarmEnv)을 동일하게 사용하며, 에피소드는 전체 작물 처리 완료 또는 최대 스텝 수 도달($\text{max_steps} = 540$) 시 종료된다.

본 문제의 행동 공간은 이산적이며 매 스텝마다 유효 행동 집합이 가변적이므로 동적 행동 마스킹의 적용이 필요하다. 무효 행동을 사후 패널티로만 억제할 경우, 초기 탐색 과정에서 막대한 샘플 낭비가 발생하여 정책 수렴이 크게 지연될 수 있다. 따라서 이산 행동 공간과 동적 마스킹을 동시에 지원하는 Maskable PPO 알고리즘을 채택하였다. 모델

학습은 4개의 병렬 환경에서 총 500,000 스텝 동안 수행되었으며, 주요 하이퍼파라미터는 Learning rate = 3×10^{-4} , n_steps = 2048, Batch size = 64로 설정하였다.

2.3.2 Step 2·3 — 이산 계층형 RL (HL PPO / DQN + LL MaskablePPO)

단일 정책은 "어느 레인을 방문할지"라는 전역 계획(Global planning)과 "레인 안에서 어떻게 움직일지"라는 국소 제어(Local control)을 한 정책으로 동시에 학습해야 한다. 이러한 문제를 완화하기 위해 본 연구에서는 상위 정책(High-Level Policy)과 하위 정책(Low-Level Policy)으로 구성된 계층형 구조를 도입하였다.

하위 환경(LaneExecutorEnv)은 기존 FarmEnv에 목표 레인을 나타내는 채널(ch₄)을 추가하여 총 5개의 채널로 구성된다. 따라서 관측 공간은 이를 평탄화한 900차원 벡터로 정의하였다. 하위 정책은 지정된 레인 내부에서 예찰 및 작업을 수행하며, 목표 레인의 모든 작업이 완료되면 에피소드가 종료된다. 또한 레인 단위 완수를 유도하기 위해 완료 보너스(+10.0)를 추가로 부여하였다.

상위 환경(HighLevelFarmEnv)은 하위 정책을 호출하는 메타 환경(Meta-Environment)으로 구성하였다. 상위 정책의 행동은 다음 목표 레인을 선택하는 (Discrete(5)) 공간으로 정의되며, 한 번의 상위 행동이 선택되면 하위 정책은 해당 레인의 작업이 완료될 때까지 실행된다.

Step 2와 Step 3은 동일한 계층 구조를 사용하지만, 하위 정책의 학습 조건과 상위 정책의 관측 공간 및 알고리즘 설정에는 차이가 있다. 하위 정책은 여전히 이산 이동·작업과 행동 마스킹을 모두 처리해야 하므로 Step 1과 동일하게 Maskable PPO를 사용하였다. Step 2의 하위 정책은 500,000 스텝 동안 학습하였으며, Step 3에서는 목표 도달 보상이 추가된 하위 정책을 700,000 스텝 동안 학습하였다. Step 2는 계층 구조 도입 효과를 평가하기 위해 상위 정책에 PPO를 적용하였다. 상위 관측을 레인별 완료율(5차원)로 두고, 하위 정책 실행 결과를 바탕으로 총 200,000 스텝 동안 학습하였다.

Step 3은 상위 정책의 상태 표현 확장과 Off-policy 학습을 결합한 통합 개선 단계이다. 상위 관측 상태에 현재 위치로부터 각 레인까지의 정규화된 거리 정보(5차원)를 추가하여 총 10차원으로 확장하였다. 이를 통해 상위 정책이 레인별 작업 완료율뿐만 아니라 레인 간 이동 비용까지 고려하여 다음 목표 레인을 선택할 수 있도록 하였다.

또한 상위 정책은 하위 정책 실행이 완료될 때에만 새로운 전이를 생성하므로 경험 수집 속도가 상대적으로 제한된다. 이에 따라 경험 재생(Experience Replay)을 활용하여 과거 전이를 반복 학습할 수 있는 Off-policy 기반의 DQN을 적용하였다. 상위 행동 공간은 5개의 레인 중 하나를 선택하는 저차원 이산 공간으로 구성되며, 최종 실행 스크립트에서는 총 300,000 스텝 동안 학습하였다. 본 단계의 목적은 계층 구조 위에서 상태 표현 확장과 Off-policy 학습이 전역 탐색 성능에 미치는 영향을 평가하는 데 있다.

Step 1부터 Step 3까지의 환경 구성, 관측 공간, 행동 공간 및 주요 학습 설정은 Table 5에 정리하였다.

Table 5. Step 1~3의 환경 구성 및 최종 학습 설정

구분	Step 1	Step 2 하위 계층	Step 2 상위 계층	Step 3 하위 계층	Step 3 상위 계층
환경	FarmEnv	LaneExecutorEnv	HighLevelFarmEnv	LaneExecutorEnv	HighLevelFarmEnv
역할	경로 탐색 + 작업 수행	목표 레인 내부 이동 및 작업	다음 목표 레인 선택	목표 레인 내부 이동 및 작업	다음 목표 레인 선택
알고리즘	Maskable PPO	Maskable PPO	PPO	Maskable PPO	DQN
관측 공간	4 채널 평탄화 벡터, 720 차원	5 채널 평탄화 벡터, 작업 완료율, 900 차원	5 차원	5 채널 평탄화 벡터, 작업 완료율 + 거리, 900 차원	10 차원
행동 공간	Discrete(7) + Masking	Discrete(7) + Masking	Discrete(5)	Discrete(7) + Masking	Discrete(5)
학습 스텝	500,000	500,000	200,000	700,000	300,000
비고	Flat baseline	기본 하위 정책	On-policy 상위 정책	목표 도달 보상 추가	거리 관측 + Off-policy 상위 정책

2.3.3 Step 4 — 연속 환경 + 연속 제어 (SAC / TD3 / TQC 비교)

본 단계는 이산 격자 기반 환경을 연속 공간으로 확장하였을 때 강화학습 정책이 실제 로봇 제어와 유사한 조건에서도 효과적으로 동작할 수 있는지 검증하고, 대표적인 연속 제어 알고리즘들의 성능을 비교·분석하기 위한 단계이다.

연속 환경(ContinuousFarmEnv)은 2.1절에서 정의한 온실 재배단 구조, 작업 종류 및 POMDP 특성을 유지하되 상태 및 행동 공간을 연속 공간으로 재정의하였다. 에이전트의 위치는 2차원 실수 좌표로 표현되며, 행동 공간은 2차원 속도 벡터로 구성된다. 예찰, 수확 및 방제 작업은 별도의 행동 입력 없이 작물과의 거리가 설정된 임계값 이하가 되면 자동 수행되도록 설계하였다. 충돌 여부는 로봇을 원 형태의 물체로 단순화하여 장애물과 겹치면 충돌로 판단하였다.

관측 공간은 고차원 격자 표현 대신 의사결정에 필요한 핵심 정보만을 추출한 28차원 벡터로 구성하였다. 관측은 에이전트의 위치 및 방향 정보(4차원), 네 방향의 이동 가능 여부를 나타내는 내비게이션 플래그(4차원), 그리고 가장 가까운 미처리 작물 5개의 상대 상태 정보(20차원)로 구성된다. 가장 가까운 5개 작물만 사용하는 이유는 관측 차원의 과

또한 증가를 방지하면서도 국소 의사결정에 필요한 주변 정보를 충분히 제공하기 위함이다. 또한 내비게이션 플래그는 물리적으로 이동이 불가능한 방향을 명시적으로 제공하여 연속 환경에서 빈번하게 발생하는 불필요한 충돌과 경계면 탐색을 줄이는 역할을 한다.

연속 공간에서는 탐색 영역이 크게 증가하여 희소 보상 문제가 더욱 심화될 수 있다. 이를 완화하기 위해 잠재 기반 보상 형태화(Potential-Based Reward Shaping)를 적용하였다. 잠재 함수는 가장 가까운 미처리 작물까지의 거리 정보를 기반으로 정의하였으며, 상태 가치 차분 형태의 보상 구조를 사용하여 정책 불변성(Policy Invariance)을 유지하면서 탐색 효율을 향상시키고자 하였다.

연속 환경 클래스는 스폰 난이도를 단계별로 조절할 수 있는 기능을 지원한다. 다만 최종 학습 스크립트에서는 curriculum_level을 2로 고정하여 모든 에피소드가 헤드랜드에서 시작하도록 설정하였다. 따라서 최종 성능은 점진적 커리큘럼 효과가 아니라, 전체 환경에서 28차원 관측 단순화와 내비게이션 플래그를 적용한 조건의 결과로 해석한다.

연속 행동 공간에서는 Maskable PPO나 DQN과 같은 이산 제어 알고리즘을 직접 적용할 수 없으므로, 연속 행동 공간을 지원하는 Off-policy 기반 알고리즘을 적용하였다. 또한 본 환경은 넓은 연속 탐색 공간과 희소 보상 특성을 동시에 가지므로, 정책의 탐색 방식과 가치 함수 추정 안정성이 성능에 큰 영향을 미칠 것으로 예상된다. 이에 따라 본 연구에서는 탐색 전략과 가치 함수 추정 방식이 서로 다른 SAC, TD3, TQC 세 가지 알고리즘을 비교 대상으로 선정하였다. 각 알고리즘의 정책 특성과 학습 설정은 Table 6에 정리하였다.

SAC(Soft Actor-Critic)는 최대 엔트로피(Maximum Entropy) 목적 함수를 활용하는 확률론적 정책 기반 알고리즘이다. 정책이 보상만 최대화하는 것이 아니라 행동 분포의 엔트로피도 함께 유지하도록 학습하기 때문에, 초기 탐색이 중요한 희소 보상 환경에서 유리할 것으로 판단하였다. 따라서 SAC는 본 연속 환경에서 확률론적 탐색 성능을 확인하기 위한 주요 비교 모델로 사용하였다.

TD3(Twin Delayed DDPG)는 결정론적 정책(Deterministic Policy)을 기반으로 하는 대표적인 연속 제어 알고리즘이다. 행동 선택은 결정론적 정책에 탐색 노이즈를 추가하는 방식으로 이루어지며, 이중 Q 네트워크와 지연 정책 업데이트를 통해 DDPG 계열에서 발생하기 쉬운 Q값 과대추정 문제를 완화한다. 본 연구에서는 TD3를 확률론적 정책인 SAC와 대비되는 결정론적 연속 제어 기준 모델로 사용하였다.

TQC(Truncated Quantile Critics)는 SAC와 같이 확률론적 정책을 사용하지만, 가치 함수를 단일 Q값이 아니라 분포형 Quantile로 추정한다. 또한 상위 분위수 일부를 제거하여 Q값 과대평가를 억제한다. 본 환경에서는 희소 보상과 긴 탐색 과정으로 인해 가치 함수 추

정이 불안정해질 수 있으므로, TQC를 통해 분포형 가치 함수와 과대추정 억제 기법이 연속 농업 로봇 환경에서 성능 개선에 기여하는지 확인하고자 하였다.

각 단계의 성공 종료 조건과 최대 스텝 수는 Table 7에 정리하였다.

Table 5. Step 4에서 사용한 연속 제어 알고리즘의 학습 설정

	SAC	TD3	TQC
정책 형태	Stochastic, 최대 엔트로피	Deterministic + 탐색 노이즈	Stochastic, distributional Q
학습 스텝	2,000,000	1,500,000	1,500,000
배치 사이즈	256	4,096	4,096
학습 주기	1	16	16

Table 6 학습 단계별 종료 기준 및 최대 스텝 수

모델	성공 종료 기준	max_steps
Step 1 FarmEnv	전체 작물 완료	540 (= $H \times W \times 3$)
Step 2-3 LaneExecutorEnv	목표 레인 작물 완료	180 (하위 정책 1 회 실행 기준)
Step 4 연속 환경	전체 작물 완료	1200

3. 실험 결과 및 분석

이 절에서는 최종 정책의 정량 결과와 개발 과정에서 얻은 보조 결과를 제시한다.

3.1 단계별 종합 성능 비교

Table 8은 네 단계의 최종 성능을 요약한다. 성공률은 제한 스텝 안에 전체 작업을 마친 비율이며, 커버리지는 종료 시점에 처리된 작물의 비율이다. 평균 스텝은 같은 환경 안에서 정책 효율을 비교하는 지표로 사용했다. 평균 레인 방문은 상위 정책의 목표 레인 선택 횟수를 기준으로 계산하였으므로, Flat 구조인 Step 1과 Step 4에는 적용하지 않았다. 또

한 Step 4는 상태·행동 공간과 최대 스텝 수가 Step 1~3과 다르므로, 이산 환경과 연속 환경의 스텝 수를 직접적인 우열로 해석할 순 없다.

Table 8. 단계별 최종 성능 비교

단계	알고리즘	성공률	커버리지	평균 스텝	평균 레인 방문
Step 1	MaskablePPO (Flat)	96%	99.8%	147 ± 80	— (flat)
Step 2	Hierarchical PPO	98%	99.8%	198 ± 349	2.3
Step 3	Goal LL + 거리 인식 DQN HL	100%	100%	141 ± 3	2.0
Step 4	SAC + 단순화 관측	96.7%	99.9%	157 ± 194	— (flat)

Table 8에서 모든 단계는 높은 커버리지를 보였으나, 평균 스텝과 표준편차에서는 단계별 차이가 나타났다. 특히 이산 환경에서는 Step 3이 가장 낮은 평균 스텝과 변동성을 보였고, 연속 환경에서는 SAC 기반 최종 구성이 높은 성공률과 커버리지를 기록하였다.

3.2 이산 환경 결과 분석: Step 1~3

Step 1의 Flat Maskable PPO는 단일 정책만으로 경로 탐색과 작업 수행을 동시에 학습하였다. 학습된 정책은 한 레인을 따라 이동한 뒤 헤드랜드를 통해 인접 레인으로 전환하는 지그재그 순회 패턴을 보였다. 이 경로 패턴은 보상이나 규칙으로 직접 지정하지 않았음에도 나타났다는 점에서, 단일 정책이 누적 보상 신호만으로도 작업 순서를 학습할 수 있음을 보여준다. 다만 일부 에피소드에서는 최대 스텝에 도달하여 종료되었으며, 이는 단일 정책이 전역 계획과 국소 제어를 동시에 학습하는 과정에서 실행 일관성이 떨어질 수 있음을 시사한다.

Step 2에서는 레인 선택과 레인 내부 작업을 상·하위 정책으로 분리하였다. 그러나 계층 구조를 도입하는 것만으로 효율이 자동적으로 개선되지는 않았다. 상위 정책의 관측이 레인별 완료율에 한정되어 있었기 때문에, 다음 목표 레인을 선택할 때 이동 비용이나 현재 위치와의 관계를 충분히 고려하기 어려웠던 것으로 해석된다. 즉, 계층 구조 자체보다 상위 정책에 어떤 정보를 제공하는지가 성능에 더 큰 영향을 미친 것으로 보인다.

Step 3에서는 Step 2와 동일한 계층형 구조를 유지하되, 상위 관측에 레인별 거리 정보를 추가하고 상위 알고리즘을 DQN으로 변경하였다. 또한 하위 정책의 목표 도달 보상과 학습량도 함께 조정하였다. 그 결과 Step 3 최종 구성은 이산 환경 중 가장 안정적인 성능을 보였다.

Target lane 구조는 실제 농기계가 가까운 재배단을 순차적으로 방문하며 작업하는 방식과 유사한 계층형 전략을 학습할 수 있는지 확인하기 위해 도입하였다. 그러나 최종 Step 3 정책의 평균 레인 방문 수는 2.0회로, 기대했던 5개 레인의 명시적 순차 방문 패턴은 나

타나지 않았다. 이는 하위 정책이 목표 레인으로 이동하는 과정에서 주변 작물까지 함께 처리했기 때문으로 해석된다. 따라서 본 결과는 에이전트가 사람이 기대한 작업 절차를 그대로 재현하기보다, 현재 보상 구조에서 더 효율적인 행동을 학습했을 가능성을 보여 준다. 또한 Greedy 규칙과 동일한 집계 성능을 보였다는 점에서, 현재 환경의 상위 의사 결정은 강화학습이 반드시 필요할 만큼 복잡하지 않았을 가능성도 있다.

3.3 연속 환경 결과 분석: Step 4

Step 4에서는 이산 이동과 명시적 작업 행동을 연속 속도 제어와 거리 기반 자동 작업으로 변환하였다. 최종 관측은 로봇 위치와 방향, 네 방향의 이동 가능 여부를 나타내는 네 비게이션 플래그, 그리고 가장 가까운 미처리 작물 5개의 정보로 구성하였다. Table 9는 동일한 ContinuousFarmEnv에서 SAC, TD3, TQC의 최종 학습 설정을 적용했을 때의 평가 결과를 나타낸다. 최종 구성에서는 SAC가 96.7% 성공률과 99.9% 커버리지를 기록하였다. TD3는 전체 작업을 완료한 에피소드가 없었고, TQC의 성공률은 3.3%에 그쳤다.

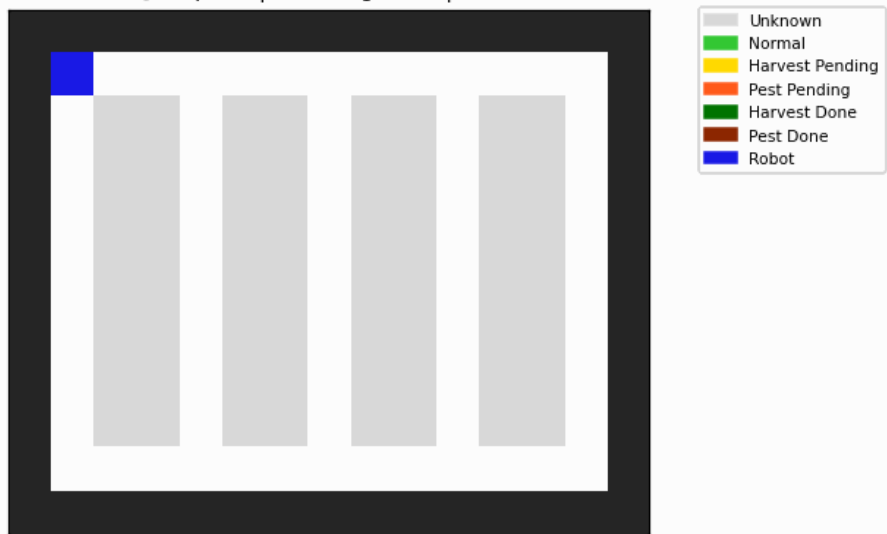
TD3와 TQC의 낮은 성공률은 연속 환경의 희소 보상과 탐색 난이도 때문으로 해석된다. TD3는 결정론적 정책에 탐색 노이즈를 더하는 방식이므로 초기 유효 trajectory를 충분히 확보하지 못했을 가능성이 있으며, TQC는 보수적인 가치 추정 구조로 인해 드문 성공 경험의 가치가 충분히 전파되지 않았을 수 있다. 반면 SAC는 최대 엔트로피 목적을 통해 행동 다양성을 유지하므로, 본 환경에서 작물에 접근하는 유효 행동을 상대적으로 더 잘 탐색한 것으로 보인다.

Table 9. 연속 환경 알고리즘별 최종 결과

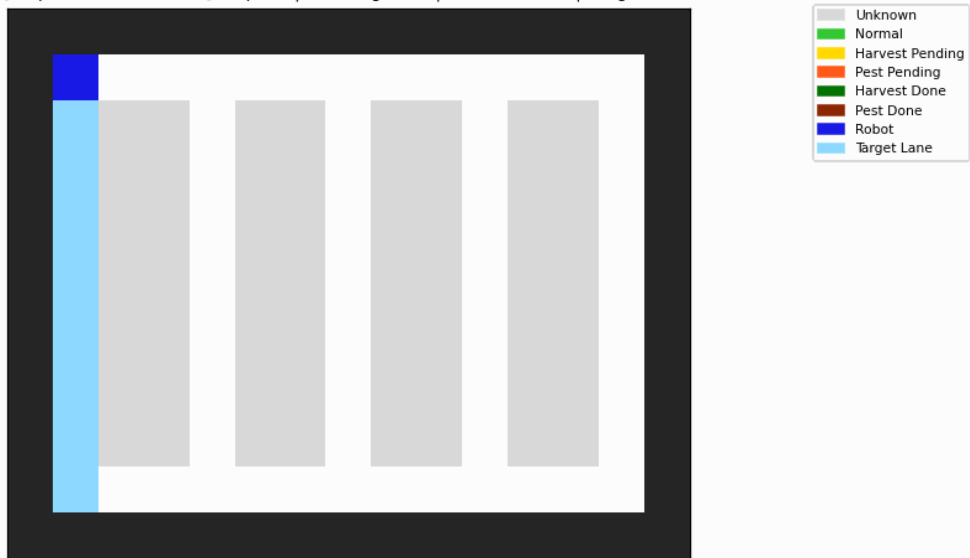
알고리즘	성공률	커버리지	평균 스텝	비고
TD3	0%	42.3%	1200 (전부 truncated)	Deterministic + 탐색 노이즈
TQC	3.3%	60.0%	1164 ± 196	Distributional Q
SAC	96.7%	99.9%	157 ± 194	최대 엔트로피

아래는 매 Step 마다의 최적 결과 gif를 보여준다

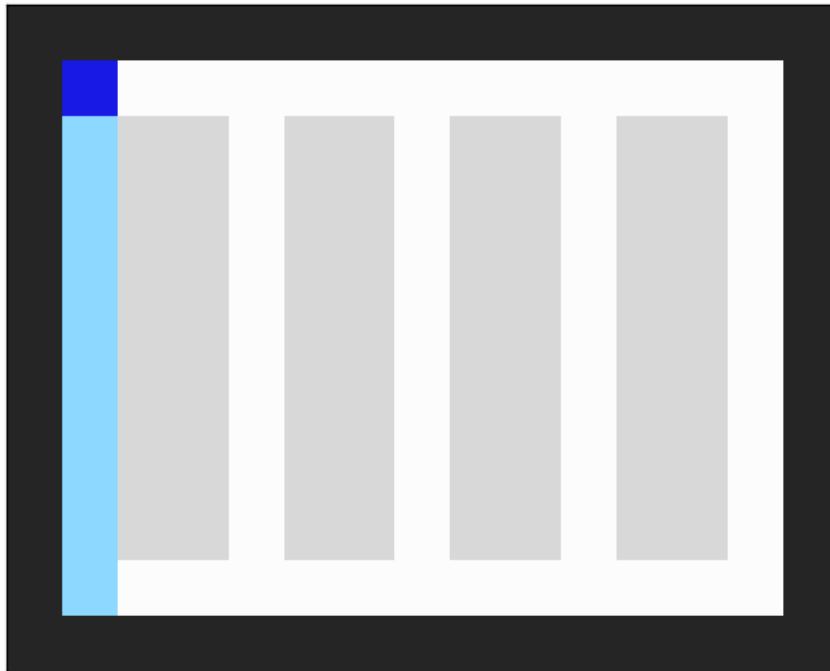
[Step1 MaskablePPO] Step 0 | Coverage 0% | Cumulative Reward +0.0



[Step2 Hierarchical PPO] Step 0 | Coverage 0% | Lane Visit #0 | Target Col 1

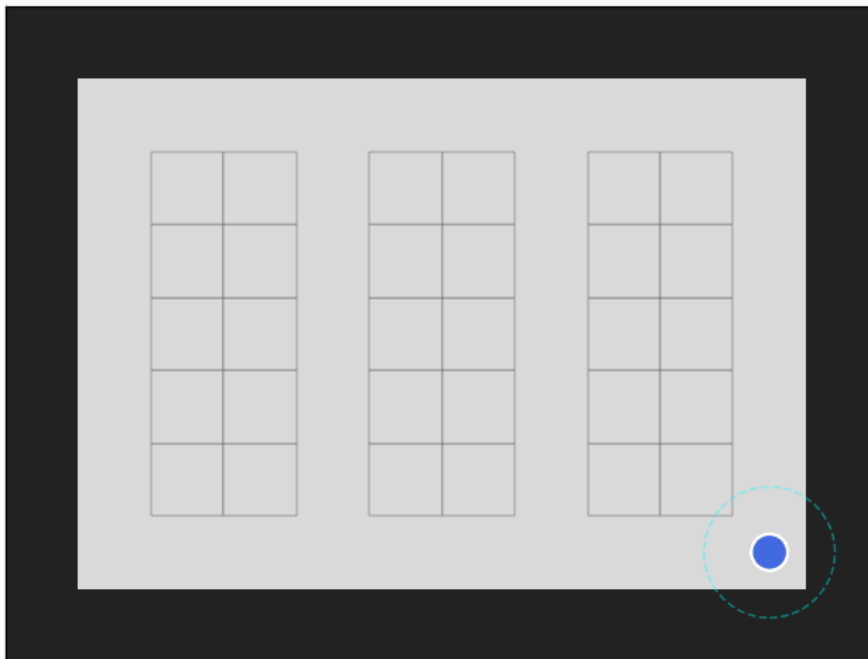


[Step3 DQN] Step 0 | Coverage 0% | Lane Visit #0 | Target Col 1



- Unknown
- Normal
- Harvest Pending
- Pest Pending
- Harvest Done
- Pest Done
- Robot
- Target Lane

[Step4 SAC+Curriculum] Step 0 | Coverage 0%



- Unknown
- Normal
- Harvest Pending
- Pest Pending
- Harvest Done
- Pest Done

3.4 보조 실험 및 개발 과정 분석

이 절에서는 최종 단계 비교를 보완하기 위해 개발 과정에서 수행한 보조 실험과 주요 시행착오를 정리한다(table 10).

Table 10. 보조 실험 및 개발 과정 요약

구분	비교 또는 변경 내용	기록된 성능	해석
마스킹 지원 정책 비교	Recurrent PPO 와 Maskable PPO 비교	0% 성공률, 0.9% 커버리지 vs 96% 성공률, 99.8% 커버리지	유효 행동이 동적으로 변하는 이산 환경에서는 행동 마스킹이 학습 안정성에 중요함
스텝 패널티 강화	매 스텝 보상 $-0.1 \rightarrow -0.3$	성공률 96% $\rightarrow$ 80%	시간 패널티를 키운다고 효율적인 경로가 자동으로 학습되지는 않음
연속 환경 관측 개선	124 차원 관측 $\rightarrow$ shaping $\rightarrow$ nav_flags $\rightarrow$ 28 차원 관측	기록된 성능 0% $\rightarrow$ 6% $\rightarrow$ 56% $\rightarrow$ 96.7%	관측 단순화와 이동 가능 정보 제공이 연속 환경 학습에 중요함
연속 계층형 RL	SAC / PPO / A2C 하위 정책 비교	41.7% / 21.7% / 16.7% 커버리지	지정 레인 처리 문제는 goal-conditioned 성격이 강해 추가 기법이 필요함

마스킹 지원 정책 비교에서는 Maskable PPO가 높은 성공률과 커버리지를 보인 반면, 마스킹을 지원하지 않는 Recurrent PPO는 거의 작업을 수행하지 못했다. 이는 본 환경처럼 매 스텝마다 유효 행동 집합이 달라지는 문제에서 무효 행동을 사후 패널티로만 처리하는 것보다, 정책 선택 단계에서 직접 제한하는 방식이 효과적일 수 있음을 보여준다. 다만 두 모델은 마스킹 여부뿐 아니라 정책 구조도 다르므로, 이 결과를 행동 마스킹만의 독립 효과로 해석하기에는 한계가 있다.

스텝 패널티를 -0.1 에서 -0.3 으로 강화한 실험에서는 성공률이 오히려 낮아졌고 평균 스텝도 줄어들지 않았다. 이는 시간 비용을 크게 부여한다고 해서 에이전트가 자동으로 더 짧은 경로를 학습하는 것은 아님을 의미한다. 본 환경에서는 과도한 시간 패널티가 탐색을 위축시켜, 필요한 우회 이동이나 추가 예찰 행동까지 회피하게 만들었을 가능성이 있다.

연속 환경은 초기 설정만으로는 학습이 제대로 이루어지지 않았다. 124차원 관측을 사용한 초기 SAC 구성은 거의 성능을 내지 못했으며, potential-based shaping, nav_flags, 가까운 미처리 작물 중심의 28차원 관측을 순차적으로 적용하면서 성능이 개선되었다. 특히 모

든 작물 정보를 고차원으로 제공하는 것보다, 현재 의사결정에 필요한 주변 정보와 이동 가능 방향을 압축적으로 제공하는 방식이 연속 제어 학습에 더 적합한 것으로 나타났다.

마지막으로, 연속 환경에 계층형 구조를 그대로 적용한 실험은 기대만큼 성공적이지 않았다. 목표 레인을 지정받은 하위 정책은 단순히 전체를 순찰하는 것보다 더 어려운 문제였기 때문이다. 따라서 연속 계층형 RL을 안정적으로 학습시키기 위해서는 목표 레인을 에이전트가 더 명확히 이해할 수 있도록 관측과 보상 구조를 다시 설계할 필요가 있다.

3.5 연구 질문에 대한 종합 답변

Q1. 이산 격자에서 Flat RL로 학습 가능한가? 가능하다. Step 1은 96% 성공률과 99.8% 커버리지를 기록했으며, 별도의 경로 규칙 없이 지그재그 순회 패턴을 학습하였다. 다만 에피소드별 스텝 변동이 커, 단일 정책이 전역 계획과 국소 제어를 동시에 학습하는 데에는 실행 안정성 측면의 한계가 있었다.

Q2. 계층 분리로 효율이 향상되는가? 계층 분리만 적용한 Step 2에서는 효율과 안정성 측면의 명확한 개선이 확인되지 않았다. 성공률은 높았지만 평균 스텝과 변동이 증가하였다. 따라서 계층 구조가 효과를 내려면 상위 정책의 판단에 필요한 관측 정보와 안정적인 하위 정책이 함께 갖춰져야 한다.

Q3. 거리 정보를 포함한 Step 3 구성은 안정적인 정책 학습에 기여하는가? Step 3 최종 구성은 100% 성공률과 141 ± 3 스텝을 기록하여 이산 환경 중 가장 안정적인 성능을 보였다. 그러나 거리 관측뿐 아니라 상위 알고리즘, 하위 보상, 학습량도 함께 변경되었으므로, 거리 정보의 독립 효과라고 보기는 어렵다.

Q4. 연속 환경으로 확장할 수 있으며 어떤 알고리즘이 적합한가? 연속 환경에서도 최종 SAC 구성으로 96.7% 성공률과 99.9% 커버리지를 얻었다. 현재 설정에서는 SAC 기반 구성이 가장 안정적으로 작동했지만, SAC, TD3, TQC는 학습 예산과 업데이트 설정이 서로 다르므로 알고리즘 간 공정한 우열 판단을 위해서는 동일 조건의 반복 비교가 필요하다.

4. 논의 및 향후 연구

4.1 주요 발견

결과를 종합하면, 본 문제에서는 알고리즘 선택만큼이나 환경 구조와 관측 설계가 중요했다. 초기 1열 재배단 구조에서는 한 레인만 통과해도 대부분의 작업을 처리할 수 있어 레인 순서 결정 문제가 뚜렷하게 나타나지 않았다. 이후 현실성을 고려하여 2열 재배단 구조로 확장한 뒤에야 상위 정책이 어느 레인을 먼저 선택할 것인지에 대한 문제가 명확해졌다. 연속 환경에서도 모든 작물 정보를 그대로 관측으로 제공한 구성보다, 이동 가능 방향과 가까운 미처리 작물 정보를 중심으로 관측을 단순화한 구성이 더 높은 성능을 보

였다. 이는 강화학습에서 단순히 많은 정보를 제공하는 것보다, 에이전트가 실제 의사결정에 사용할 수 있는 형태로 정보를 구성하는 것이 중요함을 보여준다.

계층 구조의 효과 역시 상·하위 정책을 분리하는 것만으로 보장되지는 않았다. Step 2는 상위 정책과 하위 정책을 분리했지만 Step 1보다 안정적이지 않았다. 반면 Step 3 최종 구성은 낮은 변동으로 전체 작업을 완료하였다. 이 결과는 계층형 RL을 사용할 때 상위 정책의 관측 정보와 하위 정책의 목표 정의를 함께 설계해야 함을 보여준다. 동시에 Step 3의 상위 정책은 Greedy 규칙과 동일한 집계 성능을 보였으므로, 현재와 같이 상위 의사결정이 단순한 환경에서는 규칙 기반 선택기가 충분할 가능성도 확인되었다.

행동 제약을 학습 과정에 반영하는 방식도 성능과 관련이 있었다. 이산 환경에서는 행동 마스킹을 지원하는 구성이 높은 성능을 보였고, 연속 환경에서는 이동 가능 방향을 나타내는 `nav_flags`를 포함한 구성이 성능 개선과 연결되었다. 이는 무효 행동을 사후 페널티로만 처리하기보다, 정책이 판단할 수 있는 형태로 제약 정보를 직접 제공하는 방식이 본 문제에 더 적합할 수 있음을 시사한다. 다만 이러한 결과는 독립적인 ablation으로 검증된 것은 아니므로, 개별 요소의 효과를 단정하기에는 한계가 있다.

연속 제어 환경으로의 확장 가능성도 확인되었다. 최종 SAC 구성은 높은 성공률과 커버리지를 기록하여, 이산 격자 환경이 아닌 연속 상태·행동 공간에서도 강화학습 기반 작업 수행 정책을 학습할 수 있음을 보였다. 그러나 SAC, TD3, TQC는 학습 예산과 업데이트 설정이 서로 달랐기 때문에 알고리즘 자체의 순수한 우열을 판단하기는 어렵다. 따라서 본 결과는 SAC가 일반적으로 가장 우수하다는 의미가 아니라, 현재 환경 설계와 최종 학습 설정에서 SAC 기반 구성이 가장 안정적으로 작동했다는 결과로 해석하는 것이 타당하다.

4.2 한계 및 향후 연구

본 연구의 가장 큰 한계는 주요 비교 중 일부가 하나의 요소만 바꾼 통제 실험이 아니라는 점이다. Step 2와 Step 3 사이에는 상위 관측의 거리 정보 추가, 상위 알고리즘 변경, 하위 보상 수정, 학습량 차이가 함께 존재한다. 또한 SAC, TD3, TQC 비교에서도 학습 스텝, 배치 크기, 학습 주기가 서로 다르다. 강화학습 알고리즘마다 적합한 하이퍼파라미터가 다를 수 있다고 판단하여 각 알고리즘별 최종 설정을 사용했지만, 알고리즘 간 공정한 비교를 위해서는 동일한 평가 예산과 통제된 설정이 필요했을 것 같다.

두 번째 한계는 환경의 단순화이다. 현재 환경은 단일 로봇과 정적인 작물 상태를 가정하며, 실제 농업 환경에서 발생할 수 있는 센서 오차, 미끄러짐, 통신 지연, 동적 장애물 등을 포함하지 않는다. 또한 작물 상태가 시간에 따라 변화하거나, 여러 로봇이 동시에 작업하는 상황도 고려하지 않았다. 후속 연구에서는 이러한 요소를 추가하여 실제 환경과의 차이를 줄일 필요가 있다.

연속 계층형 RL에 대한 추가 연구도 필요하다. 본 연구에서는 Flat SAC 정책이 연속 환경에서 높은 성능을 보였지만, 목표 레인을 지정받는 연속 하위 정책은 기대만큼 안정적으로 학습되지 않았다. 이는 연속 계층형 제어가 단순 순차 문제가 아니라 특정 목표를 조건으로 행동해야 하는 goal-conditioned 문제에 가깝기 때문으로 해석된다. 따라서 후속 연구에서는 HER와 같은 goal-conditioned 학습 기법을 적용하거나, 목표 레인을 에이전트가 더 명확히 이해할 수 있도록 관측 및 보상 구조를 다시 설계할 필요가 있다.

마지막으로, 실제 로봇 적용을 위한 검증이 필요하다. 본 연구는 시뮬레이션 환경에서 정책을 학습하고 평가하였으므로, 실제 로봇 플랫폼에서 동일한 성능이 보장된다고 보기는 어렵다. 후속 단계에서는 학습된 연속 제어 정책의 속도 출력을 ROS2의 `cmd_vel`과 같은 실제 로봇 제어 명령으로 연결하고, 실제 주행 환경에서 충돌 안전성, 작업 성공률, 경로 효율성을 검증해야 한다.

4.3 프로젝트 수행 과정에서의 학습점

본 프로젝트를 수행하면서 강화학습 이론을 이해하는 것과 이를 실제 코드로 구현하는 것은 상당히 다른 문제라는 점을 느꼈다. 이론적으로는 상태, 행동, 보상, 정책, 가치 함수의 개념을 이해하더라도, 이를 커스텀 환경 안에서 오류 없이 동작하도록 구현하고 학습 가능한 형태로 만드는 과정은 훨씬 어려웠다. 코딩 에이전트의 도움을 받았음에도 환경 클래스, 관측 공간, 보상 구조, 종료 조건, 학습 스크립트를 서로 맞추는 과정에서 많은 시행착오가 있었다.

특히 강화학습은 일반적인 지도학습 모델처럼 데이터셋을 넣고 바로 결과를 확인하는 방식이 아니라, 에이전트가 환경과 상호작용하면서 데이터를 직접 수집하고 정책을 개선한다. 이 때문에 학습 시간이 예상보다 오래 걸렸고, 설정 하나를 바꾼 뒤 결과를 확인하기까지 많은 시간이 필요했다. 또한 성능이 낮게 나왔을 때 그 원인이 알고리즘 문제인지, 보상 설계 문제인지, 관측 정보 부족인지, 단순한 구현 오류인지 구분하기 어려웠다.

그럼에도 직접 환경을 만들고 여러 알고리즘을 적용해 보면서 강화학습에 대한 이해를 더 깊게 할 수 있었다. 특히 좋은 알고리즘을 선택하는 것만큼이나 에이전트가 학습할 수 있는 문제 형태로 환경을 정의하는 과정이 중요하다는 점을 체감하였다. 이번 프로젝트는 강화학습을 이론적으로 공부하는 데서 그치지 않고, 실제 문제에 적용하는 과정에서 발생하는 어려움과 한계를 직접 경험할 수 있었던 의미 있는 시간이었다.